Laboratorium 2: implementacja systemu…

- System umożliwia tworzenie powiadomień z określną treścią, kanałem (push, e-mail), strefą czasową i odbiorcą.
- System pozwala użytkownikom planować symulację wysyłki powiadomień na określony moment w przyszłości, umożliwiając ustawienie dokładnej daty i godziny symulowanego dostarczenia wiadomości.
- Kanały wysyłki wiadomości działają niezależnie. Obsługa wysyłki kanałów musi odbywać się na osobnej instancji serwera.
- System gwarantuje, że każda wiadomość zostanie dostarczona dokładnie jeden raz do danego użytkownika (brak duplikatów i brak pominięć). W przypadku problemów odejmie trzy próby dostarczenia wiadomości.
- Utworzone powiadomienia są przechowywane przez system do momentu wysłania, niezależnie od bieżącego obciążenia usługi.
- Proces wysyłki powiadomień odbywa się niezależnie od momentu ich utworzenia, w sposóbu porządkowany i niezawodny.

Laboratorium 2: implementacja systemu…

- System umożliwia ustawienie priorytetów powiadomień (nisko / wysoki). Powiadomienia z wysokim priorytetem powinny mieć większą szanse dotrzeć pierwsze.

- Mechanizm wymuszenia przesłania natychmiast oraz anulowania wysyłki wiadomości która została zaplanowana.

- System uwzględnia lokalne strefy czasowe użytkowników i unika symulowania wysyłania powiadomień w nieodpowiednich porach (np. symulacja dostarczenia wiadomości poza nocnymi godzinami lokalnymi użytkownika).

Java – Spring Boot + RabbitMQ + Quartz Scheduler

- Spring Boot – rdzeń aplikacji (zarządzanie usługami, DI).
- Spring AMQP (RabbitMQ) – niezależne kanały wysyłki.
- Quartz Scheduler – planowanie dostarczenia powiadomień.
- Spring Data JPA (PostgreSQL) – trwałe przechowywanie powiadomień.
- Spring Retry – ponowne próby wysyłki wiadomości.

Java – Spring Boot + RabbitMQ + Quartz Scheduler

```java
@Entity
public class Notification {
    @Id @GeneratedValue
    private Long id;
    private String recipient;
    private String channel; // "EMAIL" or "PUSH"
    private String content;
    private ZonedDateTime scheduledTime;
    private boolean sent = false;
}
```

```java
@Service
public class EmailSender {

    @RabbitListener(queues = "email.queue")
    @Retryable(maxAttempts = 3)
    public void sendEmail(Notification n) {
        try {
            // wysyłka e-mail przez JavaMailSender
            mailSender.send(...);
            n.setSent(true);
        } catch (Exception e) {
            throw e; // automatyczny retry
        }
    }
}
```

```java
@Service
public class NotificationScheduler {

    @Autowired
    private NotificationRepository repository;
    @Autowired
    private RabbitTemplate rabbitTemplate;

    @Scheduled(fixedRate = 60_000) // co minutę
    public void processPending() {
        List<Notification> ready = repository.findReady(ZonedDateTime.now());
        ready.forEach(n -> rabbitTemplate.convertAndSend("notification.exchange",
n.getChannel(), n));
    }
}
```

Python – FastAPI + Celery + Redis/RabbitMQ + SQLAlchemy

- FastAPI – REST API do tworzenia powiadomień i planowania symulacji.
- Celery – obsługa zadań asynchronicznych i kolejek kanałów.
- Redis / RabbitMQ – brokery i kolejki komunikatów.
- SQLAlchemy + PostgreSQL – trwałość danych.
- tenacity / Celery retries – mechanizm ponowień.

Python – FastAPI + Celery + Redis/RabbitMQ + SQLAlchemy

```python
from sqlalchemy import Column, Integer, String, DateTime, Boolean
from database import Base

class Notification(Base):
    __tablename__ = "notifications"
    id = Column(Integer, primary_key=True)
    recipient = Column(String)
    channel = Column(String)  # 'email' or 'push'
    content = Column(String)
    scheduled_time = Column(DateTime)
    sent = Column(Boolean, default=False)
```

```python
from fastapi import FastAPI
from tasks import send_notification

app = FastAPI()

@app.post("/notifications")
def create_notification(data: dict):
    notif = Notification(**data)
    session.add(notif)
    session.commit()
    send_notification.apply_async((notif.id,), eta=notif.scheduled_time)
    return {"message": "Notification scheduled"}
```

```python
from celery import Celery
from tenacity import retry, stop_after_attempt

app = Celery('notifier', broker='redis://localhost:6379/0')

@retry(stop=stop_after_attempt(3))
def deliver_email(recipient, content):
    # logika np. z biblioteką smtplib lub sendgrid
    pass

@app.task
def send_notification(notification_id):
    notif = get_from_db(notification_id)
    if notif.channel == "email":
        deliver_email(notif.recipient, notif.content)
    elif notif.channel == "push":
        deliver_push(notif.recipient, notif.content)
    notif.sent = True
    save_to_db(notif)
```

C# – .NET 8 + Hangfire + MassTransit + EF Core

- ASP.NET Core Web API – tworzenie powiadomień.
- Hangfire – planowanie i wykonywanie zadań o określonej godzinie.
- MassTransit (z RabbitMQ) – routing zadań do niezależnych instancji kanałów.
- Entity Framework Core (SQL Server lub PostgreSQL) – trwałość danych.

C# – .NET 8 + Hangfire + MassTransit + EF Core

```
public class Notification
{
    public int Id { get; set; }
    public string Recipient { get; set; }
    public string Channel { get; set; }
    public string Content { get; set; }
    public DateTimeOffset ScheduledTime { get; set; }
    public bool Sent { get; set; } = false;
}
```

```
public class EmailConsumer : IConsumer<Notification>
{
    public async Task Consume(ConsumeContext<Notification> context)
    {
        var notif = context.Message;
        await RetryPolicy.Handle<Exception>()
                         .WaitAndRetryAsync(3, attempt => TimeSpan.FromSeconds(5))
                         .ExecuteAsync(async () =>
                         {
                             await SendEmail(notif.Recipient, notif.Content);
                         });
    }
}
```

```
[ApiController]
[Route("api/notifications")]
public class NotificationController : ControllerBase
{
    private readonly IBackgroundJobClient _jobs;

    public NotificationController(IBackgroundJobClient jobs)
    {
        _jobs = jobs;
    }

    [HttpPost]
    public IActionResult Create(NotificationDto dto)
    {
        var n = MapToEntity(dto);
        db.Notifications.Add(n);
        db.SaveChanges();

        _jobs.Schedule<INotificationSender>(
            s => s.SendAsync(n.Id),
            n.ScheduledTime
        );

        return Ok();
    }
}
```